T E C H N I C A L A R C H I T E C T U R E D O C U M E N T

Arquitectura Técnica del Sistema

Monolito Modular en PHP 8 con PostgreSQL — Implementación por Etapas

Proyecto: proyecto_cuentasxcobrar

Documento: ARQ-001 · Versión 1.0

Stack: PHP 8.2 / Symfony 7 / PostgreSQL 16

Fecha: 11 de noviembre de 2026

Estado: Borrador para revisión

Índice de Contenido

Resumen Ejecutivo.....

Resumen Ejecutivo.....1

1. Introducción.....1

1.1 Propósito y alcance.....1

1.2 Relación con los documentos del proyecto.....2

1.3 Audiencia.....2

2. Visión Arquitectónica y Principios.....2

2.1 Estilo arquitectónico: monolito modular.....2

2.2 Principios de diseño.....3

2.3 Drivers de calidad.....4

3. Stack Tecnológico.....4

3.1 Visión general del stack.....4

3.2 Justificación del framework: Symfony 7.....4

3.3 Persistencia: Doctrine y PostgreSQL.....5

3.4 Jobs y mensajería: Symfony Messenger.....5

3.5 Frontend ligero.....5

4. Arquitectura en Capas.....5

4.1 Capas del sistema.....6

4.2 Reglas de dependencia.....6

4.3 Estructura de directorios por módulo.....6

5. Módulos y Límites.....7

5.1 Catálogo de módulos (contextos delimitados).....7

5.2 Contratos entre módulos.....7

5.3 Propiedad de datos y un esquema por módulo.....7

6. Integración con el ERP.....8

6.1 Patrón conector: puerto y adaptador.....8

6.2 Carga batch nocturna....8
6.3 Consulta dirigida por API....8
6.4 Idempotencia y manejo de errores....9
6.5 Mapeo configurable de campos....9
7. Seguridad....9
7.1 Autenticación y gestión de sesiones....9
7.2 Autorización por roles (RBAC)....9
7.3 Protección de secretos....10
7.4 Protecciones de aplicación....10
7.5 Auditoría de acciones....11
8. Observabilidad y Operación....11
8.1 Logging estructurado....11
8.2 Monitoreo y métricas....11
8.3 Alertas de falla de carga....12
8.4 Respaldos y recuperación....12
9. Estrategia de Pruebas....12
9.1 Pirámide de pruebas....12
9.2 Cobertura y calidad de código....13
9.3 Pruebas de carga y rendimiento....13
10. CI/CD y Despliegue....14
10.1 Pipeline de integración continua....14
10.2 Entornos....14
10.3 Despliegue y rollback....14
10.4 Estándares de código....15
11. Plan de Implementación por Etapas y Fases....15
11.1 Visión general....15
11.2 Detalle técnico por etapa....16
11.3 Fases internas de cada etapa....16
11.4 Criterios de paso entre etapas....16

12. Decisiones Técnicas Registradas (ADR)..... 17

13. Riesgos Técnicos y Mitigaciones..... 17

Nota: este índice se genera mediante códigos de campo. Tras editar el documento, haga clic derecho sobre el índice y seleccione “Actualizar campos” para refrescar los números de página.

Resumen Ejecutivo

Este documento define la arquitectura técnica del Sistema de Gestión de Cuentas por Cobrar: una aplicación web construida como monolito modular en PHP 8.2 con Symfony 7, persistencia en PostgreSQL 16 y Doctrine como capa de acceso a datos. La arquitectura está diseñada para permitir la implementación de los requisitos por etapas y fases, de modo que cada etapa entregue un incremento operable del producto sin requerir reescrituras de lo construido. La integración con el ERP se materializa en un módulo conector aislado, con carga batch nocturna idempotente y consulta dirigida por API, y la operación transaccional se sostiene sobre una base de datos organizada en un esquema por módulo.

Las decisiones de diseño siguen los principios de separación de capas con inversión de dependencias, límites de módulo estrictos con propiedad de datos, idempotencia en toda operación de carga, observabilidad mediante logging estructurado y auditoría, y seguridad por mínimo privilegio con control de acceso por rol. El documento describe el stack tecnológico, la arquitectura en capas, los módulos y sus contratos, el patrón de integración con el ERP, la seguridad, la observabilidad, la estrategia de pruebas y el pipeline de CI/CD, y cierra con el plan de implementación por etapas y fases alineado con la Especificación de Requisitos.

1. Introducción

1.1 Propósito y alcance

El propósito de este documento es establecer la arquitectura técnica de referencia del sistema, en el nivel de detalle necesario para guiar el diseño detallado y la implementación de cada etapa. Cubre las decisiones de stack, la estructura en capas y módulos, los patrones de integración, la seguridad, la observabilidad, las pruebas, el despliegue y el plan por etapas. No entra en el detalle de cada pantalla ni en el diccionario de datos, que se tratan en sus documentos específicos (SRS y diseño de base de datos), pero establece las convenciones estructurales que ambos respetan.

1.2 Relación con los documentos del proyecto

La arquitectura se alinea con la Especificación de Requisitos (SRS v1.0), cuyas etapas y módulos determinan el orden de construcción, y con el Diseño de Base de Datos del módulo de Integración (DDB-INT-001), cuyas tablas, restricciones de idempotencia y particionamiento se asumen como base de persistencia. Los requisitos no funcionales del SRS —rendimiento, disponibilidad, seguridad, auditabilidad, mantenibilidad— son los

drivers de calidad que esta arquitectura satisface y que se verifican en la Etapa 5 previa a la puesta en producción.

1.3 Audiencia

El documento está dirigido al equipo de desarrollo (líder técnico, desarrolladores PHP, DBA), al responsable de operaciones (despliegue, monitoreo, respaldos) y a la dirección del proyecto, que encontrará en el capítulo 11 el plan por etapas y los criterios de paso. Los lectores técnicos encontrarán las decisiones justificadas; los no técnicos, el contexto necesario en el resumen ejecutivo y en el plan por etapas.

2. Visión Arquitectónica y Principios

2.1 Estilo arquitectónico: monolito modular

El sistema se construye como un monolito modular: una única unidad desplegable cuya estructura interna respeta los límites de los módulos de negocio como si fueran servicios separados. Cada módulo agrupa su propia lógica de dominio, su capa de aplicación y su infraestructura, y es propietario de sus datos, que solo expone a los demás módulos mediante contratos explícitos (interfaces de dominio del módulo consumidor, nunca acceso directo a las tablas ajenas). Esta decisión responde directamente a la restricción del proyecto: para el volumen de usuarios y la naturaleza de la operación, un monolito bien modularizado es más simple de construir, desplegar y operar que una arquitectura distribuida, y preserva la opción de extraer un módulo a servicio en el futuro sin reescritura.

2.2 Principios de diseño

- Separación de capas con inversión de dependencias: el dominio no depende de la infraestructura; esta implementa las interfaces que el dominio define.
- Límites de módulo estrictos: un módulo no accede a las tablas ni a las clases internas de otro módulo; solo a través de sus contratos públicos.
- Propiedad de datos: cada módulo es dueño de su esquema de base de datos; ningún otro módulo lo modifica directamente.
- Idempotencia por defecto: toda operación de carga o de mensajería puede re-ejecutarse sin efectos secundarios.
- Observabilidad integrada: toda operación relevante deja registro estructurado y trazable.
- Seguridad por mínimo privilegio: roles separados en base de datos y en aplicación; sin borrado físico desde la interfaz.

- Configuración sobre código: umbrales, reglas y catálogos son datos administrables, no constantes en el código.

2.3 Drivers de calidad

Los requisitos no funcionales del SRS son los drivers que la arquitectura debe satisfacer. El rendimiento interactivo (RNF-PER-01) se logra con consultas optimizadas e índices selectivos; la disponibilidad (RNF-DIS-01, RNF-DIS-02) con un despliegue sencillo y operación tolerante a fallas del ERP; la seguridad y la auditoría (RNF-SEG-01 a RNF-SEG-06) con Symfony Security, RBAC, logging estructurado e inmutabilidad operativa; la mantenibilidad (RNF-MAN-01 a RNF-MAN-04) con la estructura modular, la configuración externalizada y el conector ERP aislado. La tabla 1 resume el mapeo.

Tabla 1. Drivers de calidad del SRS y respuesta arquitectónica

Driver (SRS)	Respuesta arquitectónica
Rendimiento interactivo (< 3 s)	Consultas con índices selectivos, vistas materializadas para aging, paginación, caché de consultas frecuentes (Redis opcional).
Carga batch (< 30 min)	Comando Symfony Console + UPSERT masivo idempotente en una transacción por entidad.
Disponibilidad y tolerancia a fallas	Despliegue monolítico simple; operación con última carga válida ante falla del ERP.
Seguridad y RBAC	Symfony Security, roles aplicacion + roles BD, mínimo privilegio.
Auditabilidad e inmutabilidad	Logging estructurado (Monolog), bitácora de integración, borrado lógico, sin DELETE desde la app.
Mantenibilidad	Monolito modular con límites, conector ERP aislado, configuración en catálogos.
Escalabilidad sostenida	Particionamiento de tablas históricas; diseño que triplica el volumen sin cambio de arquitectura.

3. Stack Tecnológico

3.1 Visión general del stack

El stack se eligió buscando madurez, soporte a largo plazo, alineación con las buenas prácticas de la comunidad PHP y simplicidad operativa para un equipo pequeño. Todas las decisiones son de código abierto y se despliegan en un servidor dedicado o en contenedores. La tabla 2 resume el stack; las secciones siguientes justifican las decisiones principales.

Tabla 2. Stack tecnológico del sistema

Capa	Tecnología	Versión / nota
Lenguaje y runtime	PHP	8.2 o superior, con PHP-FPM.
Framework web	Symfony	7.x (LTS).
ORM / DBAL	Doctrine	ORM 3 + DBAL, migraciones.
Base de datos	PostgreSQL	16.
Mensajería y jobs	Symfony Messenger	Carga batch, notificaciones; Redis como transporte opcional.
Plantillas	Twig	Renderizado del lado servidor.
Interactividad cliente	Alpine.js + HTMX (ligero)	Sin build pesado; API JSON cuando se requiera.
Caché y colas	Redis (opcional)	Mensajería, caché de consultas.
Servidor web	Nginx	Reverse proxy, TLS, estáticos.
Procesos	Supervisor + cron	Workers de Messenger y disparo de carga.
Calidad de código	PHPStan, Psalm, PHP CS Fixer, PHPUnit	Análisis estático, estilo, pruebas.
Dependencias	Composer	Gestión de paquetes.

3.2 Justificación del framework: Symfony 7

Symfony se elige como framework por su ajuste natural al estilo de monolito modular: su arquitectura de bundles y la Flex de directorios permiten organizar el código por módulos con fronteras claras, y ofrece componentes maduros para todas las necesidades del sistema — Security (autenticación y RBAC), Console (comandos de carga), Messenger (jobs asíncronos), Validator (validación de los datos del ERP), Doctrine (persistencia) y Twig (presentación). Su modelo de inyección de dependencias y la conformidad con los estándares PSR lo hacen mantenible y portable. Frente a alternativas, Symfony proporciona el control estructural que exige un sistema con límites de módulo estrictos, sin el acoplamiento mágico de frameworks más opinados.

3.3 Persistencia: Doctrine y PostgreSQL

Doctrine ORM mapea las entidades de dominio a las tablas de PostgreSQL definidas en el documento de base de datos, y Doctrine DBAL se utiliza directamente para las operaciones de carga masiva (UPSERT con ON CONFLICT) donde el ORM añadiría overhead innecesario. Las migraciones de Doctrine gestionan el versionado del esquema. Las consultas de lectura de cartera y reportes se implementan con DBAL o con DQL optimizado, y se apoyan en las vistas de conveniencia definidas en la base de datos para evitar lógica de agregación duplicada.

3.4 Jobs y mensajería: Symfony Messenger

Las operaciones asíncronas —carga batch, notificaciones de falla, reproceso, generación de reportes masivos— se implementan con Symfony Messenger sobre un transporte en Redis (recomendado) o en la base de datos (mínimo viable). La carga batch nocturna es un comando Symfony Console que encola el trabajo en Messenger y se ejecuta bajo Supervisor/cron; los workers procesan el lote con reintentos controlados y registro de errores. Este enfoque desacopla el disparo (cron) de la ejecución (worker) y permite reintentos y observabilidad sin lógica ad hoc.

3.5 Frontend ligero

La interfaz se renderiza con Twig del lado del servidor, complementada con Alpine.js e HTMX para la interactividad de las pantallas de gestión (filtros, modales, guardado de acciones). Esta combinación evita un pipeline de build frontend pesado y mantiene el sistema dentro del estilo monolítico tradicional, adecuado a un equipo de desarrollo PHP. Cuando un futuro módulo requiera una experiencia más rica (por ejemplo, un tablero interactivo de indicadores), se expondrán endpoints JSON consumidos por un cliente ligero, sin alterar la arquitectura.

4. Arquitectura en Capas

4.1 Capas del sistema

Cada módulo implementa internamente cuatro capas con reglas de dependencia estrictas. La capa de presentación recibe las peticiones HTTP y devuelve respuestas; la capa de aplicación orquesta los casos de uso mediante command y query handlers; la capa de dominio contiene las entidades, value objects y reglas de negocio; y la capa de infraestructura implementa la persistencia, los conectores externos y los jobs. La figura 1 muestra esta organización y los siete módulos que la replican verticalmente.



Figura 1. Arquitectura en capas del monolito modular (PHP/Symfony)

4.2 Reglas de dependencia

Las dependencias apuntan hacia el dominio: la presentación depende de la aplicación, la aplicación del dominio, y la infraestructura implementa las interfaces que el dominio define (inversión de dependencias). En ningún caso el dominio importa clases de Doctrine, Symfony framework o de otro módulo. Esta regla mantiene el dominio puro y verificable con pruebas unitarias sin infraestructura, y permite cambiar la implementación de persistencia o el conector ERP sin tocar las reglas de negocio.

4.3 Estructura de directorios por módulo

Cada módulo se materializa como un directorio con la misma estructura interna, bajo src/. Esta uniformidad facilita la navegación del código y el cumplimiento de los límites. El ejemplo muestra la estructura del módulo de Integración; los demás módulos la replican.

```
src/
└── Integracion/
    ├── Application/
    │   ├── Command/        # EjecutarCargaCommand, RegistrarErrorCommand
    │   ├── Query/          # ListarCargasQuery, ResumenCargaQuery
    │   └── DTO/
    ├── Domain/
    │   ├── Model/          # Carga, CargaDetalle, ErrorCarga (entidades)
    │   ├── Service/        # CalculadorDeContadores, ValidadorDeFactura
    │   └── Repository/     # CargaRepositoryInterface (contrato)
    ├── Infrastructure/
```

```
    │   ├── Persistence/    # DoctrineCargaRepository (implementa el contrato)
    │   ├── ErpConnector/   # ApiErpConnector, VistaBdErpConnector (adapter)
    │   └── Messaging/      # CargaBatchHandler (Messenger worker)
    └── Presentation/
        ├── Controller/     # CargaController (web + API)
        └── Twig/           # plantillas del módulo
```

El módulo SharedKernel aloja los contratos y tipos compartidos (identificadores, value objects comunes como Moneda o FechaContable, y excepciones base) que varios módulos consumen sin violar límites, pues su contenido es estable y acordado.

5. Módulos y Límites

5.1 Catálogo de módulos (contextos delimitados)

El sistema se compone de siete módulos coincidentes con los del SRS. Cada módulo es un contexto delimitado con su propio lenguaje ubicuo, sus entidades y su esquema de base de datos. La tabla 3 resume su responsabilidad y su esquema.

Tabla 3. Módulos del sistema

Módulo	Responsabilidad	Esquema BD	Etapa
SharedKernel	Contratos y tipos compartidos estables.	—	0
Seguridad	Usuarios, roles, RBAC, auditoría, configuración.	seguridad	0
Integración ERP	Conector, cargas, bitácora, errores, mapeo.	integracion	1
Cartera	Clientes, facturas, servicios, aging, niveles.	cartera	2
Cobranza	Acciones, conciliaciones, promesas, línea de tiempo.	cobranza	3
Asignación	Asignación y reasignación de clientes a especialistas.	asignacion	4
Reportes	Indicadores, aging, DSO, CEI, exportaciones.	reportes	5

5.2 Contratos entre módulos

Un módulo que necesita información de otro no accede a sus tablas ni a sus entidades internas: invoca un contrato público definido como una interfaz del módulo proveedor, implementada en su capa de aplicación. Por ejemplo, el módulo de Cobranza, al registrar una acción sobre una cuenta, consulta el saldo y el aging de la factura mediante FacturaQueryInterface del módulo Cartera, no leyendo directamente la tabla factura. Este principio mantiene el acoplamiento bajo y localiza el impacto de los cambios. SharedKernel

provee los tipos básicos que aparecen en los contratos (identificadores, Moneda, etc.) para evitar duplicar definiciones.

5.3 Propiedad de datos y un esquema por módulo

Cada módulo es propietario exclusivo de su esquema de base de datos y de las tablas que lo componen; ningún módulo escribe en el esquema de otro. Las lecturas cruzadas necesarias (por ejemplo, Reportes leyendo cartera y cobranza) se realizan mediante vistas de conveniencia o consultas explícitas a través del contrato del módulo propietario, nunca por acceso directo a sus tablas. Esta propiedad es la base de la futura extractibilidad: si un módulo se extrajera a un servicio, su esquema migraría con él sin afectar al resto.

6. Integración con el ERP

6.1 Patrón conector: puerto y adaptador

La integración con el ERP se aísla en el módulo Integración siguiendo el patrón de puerto y adaptador (hexagonal). El dominio del módulo define un puerto ErpSourceInterface con métodos como leerFacturas, leerServicios, leerPagos y leerClientes; la infraestructura proporciona uno o varios adaptadores concretos (ApiErpConnector para un ERP con API REST, VistaBdErpConnector para acceso por vistas de base de datos, ArchivoErpConnector para intercambio por archivos). La elección del adaptador se configura en runtime según integracion.conexion_erp.tipo, de modo que cambiar el mecanismo de acceso al ERP no afecta al dominio ni al resto del sistema, satisfaciendo el requisito RNF-MAN-03.

```
// Puerto definido por el dominio del módulo Integración
interface ErpSourceInterface {
    public function leerClientes(DateTimeImmutable $desde): iterable;
    public function leerFacturas(DateTimeImmutable $desde): iterable;
    public function leerServicios(DateTimeImmutable $desde): iterable;
    public function leerPagos(DateTimeImmutable $desde): iterable;
}

// Adaptador concreto (infraestructura) — selección por configuración
final class ApiErpConnector implements ErpSourceInterface { /* ... */ }
final class VistaBdErpConnector implements ErpSourceInterface { /* ... */ }
```

6.2 Carga batch nocturna

La carga batch se implementa como un comando Symfony Console (app:integracion:cargar) que crea un registro de carga en estado en_curso, encola en Messenger el trabajo por entidad (clientes, facturas, servicios, pagos) y deja que los workers lo procesen. Cada worker lee los datos del ERP mediante el adaptador configurado, valida cada registro, persiste con UPSERT idempotente (INSERT … ON CONFLICT) y registra los rechazos en la cola de errores. Al terminar, actualiza el estado y los contadores de la carga y, si hubo errores o falla,

dispara la notificación al administrador (RF-INT-06). La transacción por entidad garantiza que cada conjunto se confirma entero o se revierte entero.

6.3 Consulta dirigida por API

La consulta dirigida por API (RF-INT-07) permite refrescar el estado de una cuenta individual durante el día antes de una gestión comercial. Se expone como un endpoint interno invocado bajo demanda que crea una carga de origen api_dirigida, consulta el ERP para la factura y sus pagos, y actualiza el saldo y el estado contable mediante el mismo UPSERT idempotente. Esta consulta se realiza solo bajo solicitud explícita para no sobrecargar al ERP, y su ejecución queda registrada en la bitácora.

6.4 Idempotencia y manejo de errores

La idempotencia de la integración se garantiza en la base de datos mediante las restricciones UNIQUE sobre las claves naturales y la operación UPSERT, descritas en el documento de diseño de base de datos (sección 6.1). Los errores de validación de un registro (importe negativo, cliente inexistente, fecha incoherente) se capturan por fila, se registran en integracion.error_carga con el dato original en JSONB y no interrumpen la carga del resto del lote (RF-INT-04). El reproceso controlado (RF-INT-08) reutiliza la misma lógica sobre un rango de fechas, dejando registro del usuario que lo disparó.

6.5 Mapeo configurable de campos

El mapeo entre los campos físicos del ERP y los campos del modelo local se externaliza en la tabla integracion.mapeo_campo_erp, de modo que un cambio en los campos del ERP se resuelve con un dato administrable y no con un cambio de código. El adaptador aplica el mapeo al leer los datos del ERP y, antes de persistirlos, el validador del dominio verifica que los campos marcados como obligatorios estén presentes y sean coherentes, rechazando los registros inválidos a la cola de errores.

7. Seguridad

7.1 Autenticación y gestión de sesiones

La autenticación se implementa con Symfony Security sobre credenciales corporativas (usuario y contraseña), con almacenamiento de hash mediante algoritmos modernos (bcrypt o argon2id) y políticas de contraseña configurables (longitud mínima, complejidad, expiración). Las sesiones se gestionan con expiración por inactividad (30 minutos por defecto, parametrizable) y rotación del identificador tras el login, en cumplimiento del requisito RNF-SEG-01. La integración futura con un proveedor de identidad corporativo

(SSO via SAML u OIDC) queda habilitada por la abstracción de Symfony Security sin requerir reescritura del control de acceso.

7.2 Autorización por roles (RBAC)

La autorización sigue el modelo de roles definido en el SRS (especialista comercial, supervisor, administrador de cartera, analista de conciliación, auditor, administrador técnico). Symfony Security expresará cada rol como ROLE_ESPECIALISTA, ROLE_SUPERVISOR, etc., y los protegerá mediante voters para los accesos a recursos específicos —en particular, la restricción de que un especialista solo ve y gestiona las cuentas de los clientes que tiene asignados (RF-ASI-05). La matriz de permisos se documenta en el SRS y se implementa como voters centralizados por módulo, de modo que la verificación es uniforme y auditable.

7.3 Protección de secretos

Los secretos —credenciales del ERP, claves de la base de datos, parámetros sensibles— se gestionan como variables de entorno inyectadas en el despliegue y nunca se almacenan en el repositorio de código ni en archivos de configuración versionados. La tabla integracion.conexion_erp guarda únicamente una referencia al secreto (credenciales_ref), nunca el valor. Los logs se filtran para enmascarar campos sensibles antes de cualquier registro, de modo que ninguna credencial llegue a la bitácora (RNF-SEG-05).

7.4 Protecciones de aplicación

- TLS obligatorio en todo el tráfico (HTTPS), con redirección de HTTP y HSTS.
- Cabeceras de seguridad: X-Content-Type-Options, X-Frame-Options, Content-Security-Policy básica.
- Protección CSRF en todos los formularios y operaciones de estado (Symfony Forms y tokens).
- Validación de entrada en servidor con Symfony Validator antes de cualquier escritura.
- Consultas parametrizadas vía Doctrine DBAL/ORM; prohibida la concatenación de SQL.
- Sin borrado físico desde la aplicación: borrado lógico (activo, fecha_baja) en todas las tablas transaccionales (RNF-SEG-06).

7.5 Auditoría de acciones

Toda creación, modificación y eliminación lógica registra usuario, fecha, operación y valores anteriores y nuevos, en cumplimiento del requisito RNF-SEG-03. La auditoría se

implementa con un subscriber de Doctrine que, sobre las entidades marcadas como auditable, persiste un registro en la tabla auditoria del esquema seguridad, sin intervención del código de aplicación. La bitácora de integración complementa este registro con los eventos del proceso de carga. El auditor accede a ambas fuentes en solo lectura mediante el rol cxc_readonly.

8. Observabilidad y Operación

8.1 Logging estructurado

El sistema registra su actividad con Monolog en formato estructurado (JSON), con canales separados por módulo y por propósito (app, integracion, security, deprecations). Cada entrada incluye marca de tiempo, nivel, canal, contexto estructurado y un identificador de correlación por petición (request-id) que permite seguir una operación a través de sus componentes. Los logs se enrutan a archivos rotativos en el servidor y, en producción, a un agregador central si la organización dispone de uno. Los niveles se configuran por entorno: debug en desarrollo, info en producción, con los errores y advertencias siempre presentes.

8.2 Monitoreo y métricas

El sistema expone un endpoint interno de métricas básicas (cargas del día, estado del último job, latencia de consultas críticas, tamaño de la cola de Messenger) consumible por el sistema de monitoreo de la organización o por un dashboard operativo simple. La supervisión de cartera dispone además, dentro de la propia aplicación, de un tablero de operación que muestra la última carga, los errores pendientes y las alertas activas. No se introduce un sistema de métricas dedicado en la versión inicial; se aprovecha el existente en la organización cuando lo haya.

8.3 Alertas de falla de carga

La falla de la carga nocturna o la finalización con errores dispara una notificación al administrador técnico y a la administración de cartera (RF-INT-06), por correo electrónico y, si está configurado, por el canal de mensajería operativo. La aplicación continúa operando con la última carga válida, de modo que la falla no interrumpe la jornada. El reproceso queda a disposición del administrador desde la interfaz de administración, con registro del usuario y la causa.

8.4 Respaldos y recuperación

PostgreSQL se respalda diariamente con pg_dump completo y archivado continuo de WAL, lo que permite la recuperación a un punto en el tiempo dentro de las últimas 24 horas (RNF-

DIS-03). El procedimiento de restauración se documenta y se prueba al menos una vez por ciclo en un entorno espejo. Las particiones mensuales de bitácora y snapshot se purgan porDetach y Drop tras el período de retención, operación instantánea que no afecta la operación.

9. Estrategia de Pruebas

9.1 Pirámide de pruebas

La estrategia sigue la pirámide de pruebas: una base amplia de pruebas unitarias del dominio, una capa intermedia de pruebas de integración con base de datos de test, y un conjunto reducido de pruebas de extremo a extremo para los flujos críticos. Las pruebas unitarias del dominio son puras (sin Symfony, sin Doctrine, sin base de datos) y verifican las reglas de negocio (cálculo de aging, transiciones de estado, priorización). Las pruebas de integración utilizan una base de datos PostgreSQL de test con el esquema migrado, y verifican los repositorios y los flujos de carga idempotente. Las pruebas E2E cubren los flujos de carga, gestión comercial y asignación con usuarios de prueba.

9.2 Cobertura y calidad de código

- Cobertura objetivo del 70% en líneas y del 80% en ramas para el dominio, verificada en el pipeline.
- Análisis estático con PHPStan en nivel 6+ y Psalm, sin errores nuevos en el pipeline.
- Estilo de código con PHP CS Fixer conforme a PSR-12, verificado en el pipeline.
- Pruebas de los adaptadores ERP con dobles (mocks del puerto), sin depender del ERP real.
- Pruebas de idempotencia explícitas: dos ejecuciones de la misma carga producen los mismos datos.

9.3 Pruebas de carga y rendimiento

En la Etapa 5, previa a la puesta en producción, se ejecutan pruebas de carga al volumen proyectado (y a 3× el volumen para validar la escalabilidad, RNF-PER-05) sobre los flujos críticos: consulta de cartera con filtros, registro de acción y carga batch. Los objetivos son los del SRS: percentil 95 bajo 3 segundos en consultas interactivas y carga completa bajo 30 minutos. Los resultados se archivan como evidencia del cumplimiento de los requisitos no funcionales antes del go-live.

10. CI/CD y Despliegue

10.1 Pipeline de integración continua

Cada cambio enviado al repositorio dispara el pipeline de CI sobre GitHub Actions (o el sistema de CI de la organización). El pipeline ejecuta, en orden: instalación de dependencias con Composer, verificación de estilo con PHP CS Fixer, análisis estático con PHPStan y Psalm, suite de pruebas unitarias y de integración con una base de datos PostgreSQL de test efímera, y cálculo de cobertura. Un cambio se acepta solo si todas las etapas pasan. Las ramas se protegen y los cambios se revisan antes de fusionarse a la rama principal.

```
# Pipeline de CI (resumen de etapas)
jobs:
  quality:
    steps:
      - composer install --prefer-dist --no-interaction
      - php-cs-fixer check
      - phpstan analyse --level=6
      - psalm
      - phpunit --testsuite=unit,integration --coverage-text
  build:
    needs: quality
    steps:
      - composer install --no-dev --optimize-autoloader
      - bin/console doctrine:migrations:migrate --no-interaction  # sobre BD
de test
      - artifact: app.tar.gz
```

10.2 Entornos

Se disponen de cuatro entornos: desarrollo (local del desarrollador, con base de datos yERP simulado), pruebas (compartido, con datos de prueba anonimizados de la cartera real), staging (espejo de producción, sobre el que se ejecutan las pruebas de aceptación y de carga) y producción. La configuración de cada entorno se entrega por variables de entorno; ningún secreto se versiona. Los datos de cartera de prueba se anonimizan antes de cargarse en los entornos no productivos.

10.3 Despliegue y rollback

La topología de despliegue, mostrada en la figura 3, consta de un servidor de aplicación con Nginx como reverse proxy, PHP-FPM ejecutando la aplicación Symfony, PostgreSQL con los esquemas por módulo y un planificador (cron + Supervisor) que dispara la carga batch y los workers de Messenger. El ERP contable es un sistema externo al que el conector accede por el mecanismo configurado (API, vistas de base de datos o archivos). Redis es opcional y se incorpora como transporte de Messenger y caché de consultas frecuentes.

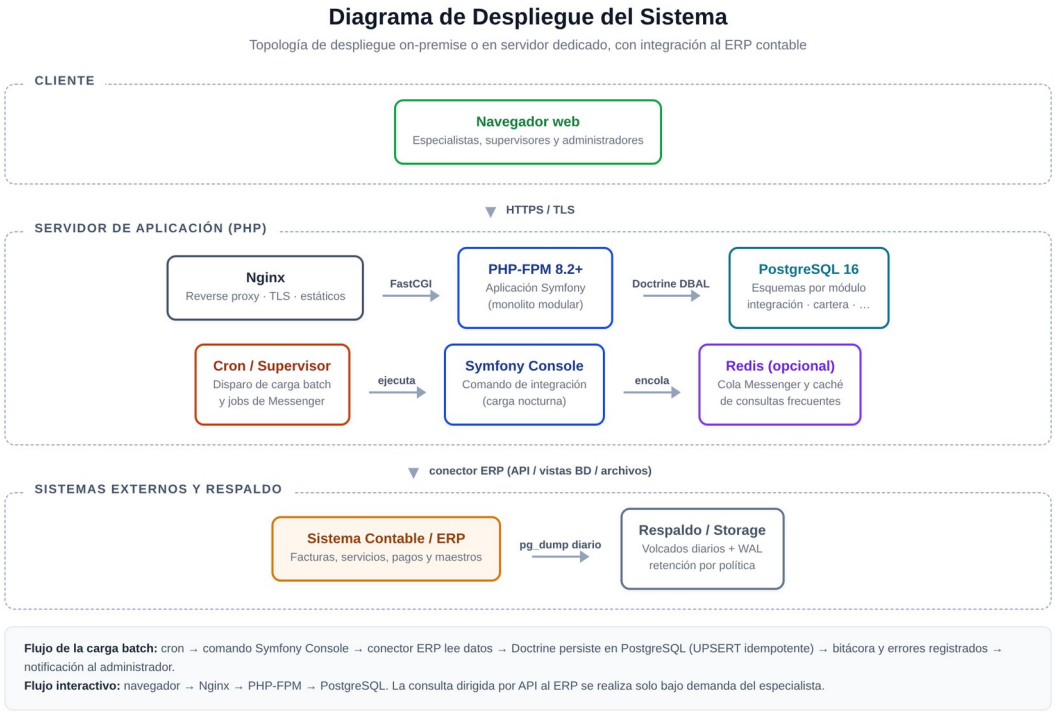


Figura 3. Diagrama de despliegue del sistema

El despliegue se realiza desde el artefacto producido por CI: se extrae el paquete en el servidor, se ejecutan las migraciones de Doctrine (idempotentes y compatibles hacia atrás, sección 8.3 del DDB) y se reinicia PHP-FPM bajo Supervisor. El despliegue incluye una verificación de salud (health check) que confirma la conectividad con PostgreSQL y con el ERP antes de declararlo exitoso. El rollback consiste en replegar el artefacto anterior y revertir la migración; por la política de compatibilidad hacia atrás de las migraciones, la versión inmediatamente anterior opera sin cambios sobre el esquema actualizado.

10.4 Estándares de código

El código se adhiere a PSR-12 para estilo y a las recomendaciones PSR-4 para autoloading, con nombres de clases en PascalCase y nombres de métodos y variables en camelCase, conforme al idioma universal de la comunidad PHP. La documentación de las clases y métodos públicos se redacta en español con bloques PHPDoc, y los identificadores del dominio usan el lenguaje ubicuo de cada módulo (Carga, Factura, PromesaDePago, NivelCobranza). El repositorio mantiene la documentación (SRS, diseño de BD, este documento) sincronizada con el código, bajo la carpeta docs/.

11. Plan de Implementación por Etapas y Fases

11.1 Visión general

La implementación se organiza en seis etapas coincidentes con los módulos del SRS, cada una entregando un incremento operable del producto. Dentro de cada etapa, el trabajo se organiza en fases internas que siguen el ciclo diseño → desarrollo → pruebas → aceptación de usuario (UAT), de modo que cada etapa cierra con una entrega validada. La figura 2 muestra el roadmap completo con las dependencias entre etapas.



Figura 2. Roadmap de implementación por etapas y fases

11.2 Detalle técnico por etapa

La tabla 4 detalla, para cada etapa, los módulos involucrados, los entregables técnicos y la dependencia con etapas previas. Las duraciones son estimaciones de referencia para un equipo pequeño y se ajustarán en el plan de proyecto.

Tabla 4. Detalle técnico por etapa

Etapa	Módulos / entregables técnicos	Dependencia
Etapa 0 — Base	Andamiaje Symfony + Doctrine; módulo Seguridad (RBAC, auditoría); esquemas y migración inicial; pipeline CI; entorno de pruebas.	Ninguna
Etapa 1 — Integración	Módulo Integración: puerto ErpSourceInterface, adaptador al ERP real, comando de carga batch con Messenger, UPSERT idempotente, bitácora y cola de errores, consulta dirigida por API.	Etapa 0
Etapa 2 — Cartera	Módulo Cartera: consulta y filtros, cálculo de aging, estados y niveles N0-N4, bandeja priorizada, ficha de cliente; enriquece el esquema cartera.	Etapa 1
Etapa 3 — Cobranza	Módulo Cobranza: registro de acciones, línea de tiempo, conciliaciones con ciclo de vida, promesas de pago, aprobaciones; esquema cobranza.	Etapa 2

Etapa	Módulos / entregables técnicos	Dependencia
Etapa 4 — Asignación	Módulo Asignación: asignación y reasignación con historial, masivas, visibilidad por cartera, carga de trabajo; esquema asignacion.	Etapa 2
Etapa 5 — Reportes y endurecimiento	Módulo Reportes: aging, DSO, CEI, exportaciones; pruebas de carga al volumen; revisión de seguridad; puesta en producción.	Etapas 3 y 4

11.3 Fases internas de cada etapa

Cada etapa se ejecuta en cuatro fases internas con entregables y verificaciones propios. Esta cadencia hace que cada etapa cierre con una entrega validada y que los riesgos se detecten temprano.

Tabla 5. Fases internas por etapa

Fase	Actividad	Entregable / verificación
F.x.1 Diseño	Diseño detallado del módulo: entidades, contratos, pantallas, DDL.	Diseño revisado y aprobado.
F.x.2 Desarrollo	Implementación por capas con pruebas unitarias del dominio.	Código en rama con CI verde.
F.x.3 Pruebas	Pruebas de integración y de los flujos críticos; corrección de defectos.	Suite de pruebas en verde.
F.x.4 Aceptación	UAT con usuarios reales sobre datos anonimizados; revisión de trazabilidad.	Acta de aceptación de la etapa.

11.4 Criterios de paso entre etapas

Una etapa se considera cerrada y habilita la siguiente cuando se cumplen, de manera demostrable, las siguientes condiciones: la suite de pruebas unitarias e de integración pasa en el pipeline con la cobertura objetivo; los criterios de aceptación definidos en el SRS para los requisitos de la etapa se verifican positivamente en el UAT; la trazabilidad requisito → implementación → prueba está completa y revisada; y el entorno de staging opera de forma estable con la nueva funcionalidad integrada. Solo en la Etapa 5 se adicionan las pruebas de carga y la revisión de seguridad como condición de la puesta en producción.

12. Decisiones Técnicas Registradas (ADR)

Las decisiones arquitectónicas significativas se registran como ADR (Architecture Decision Records) en el repositorio, bajo docs/adr/. Cada ADR documenta el contexto, la decisión, sus alternativas y sus consecuencias. La tabla 6 resume los ADR iniciales; el detalle de cada uno vive en su archivo correspondiente y evoluciona con el proyecto.

Tabla 6. Decisiones técnicas registradas

ADR	Decisión	Justificación resumida
ADR-001	Monolito modular frente a microservicios.	Simplicidad de construcción y operación para el volumen previsto; preserva la extractibilidad futura.
ADR-002	Symfony 7 como framework.	Soporte a módulos con límites, componentes maduros (Security, Console, Messenger, Doctrine), LTS.
ADR-003	PostgreSQL 16 como único gestor.	Idempotencia con UPSERT, particionamiento declarativo, JSONB para contexto de errores.
ADR-004	Puerto y adaptador para el conector ERP.	Aislar el mecanismo de acceso al ERP; cambiarlo sin afectar al dominio.
ADR-005	Doctrine ORM + DBAL (UPSERT nativo para cargas).	ORM para CRUD; DBAL para cargas masivas idempotentes.
ADR-006	Symfony Messenger para jobs.	Reintentos y observabilidad sin lógica ad hoc; transporte Redis o BD.
ADR-007	Borrado lógico e inmutabilidad operativa.	Cumplir trazabilidad e inmutabilidad (RNF-SEG-03/06).
ADR-008	Twig + Alpine.js + HTMX para el frontend.	Evitar un build frontend pesado; mantener el equipo en PHP.
ADR-009	Migraciones Doctrine compatibles hacia atrás.	Despliegues sin tiempo de incautación y rollbacks seguros.
ADR-010	CI en GitHub Actions con análisis estático y pruebas.	Calidad de código garantizada antes de fusionar.

13. Riesgos Técnicos y Mitigaciones

La tabla 7 identifica los riesgos técnicos más relevantes de la arquitectura y las mitigaciones previstas. Los riesgos se revisan al inicio de cada etapa y se actualizan conforme se conoce más del sistema contable de origen y de los datos reales de la cartera.

Tabla 7. Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación
El ERP no expone API y solo permite acceso por vistas/archivos.	Medio	El patrón puerto-adaptador aísla el mecanismo; se implementa el adaptador que corresponda sin tocar el dominio.
Variabilidad de los campos del ERP entre versiones.	Medio	Mapeo configurable en integracion.mapeo_campo_erp; el adaptador aplica el mapeo en runtime.
Crecimiento de tablas históricas (bitácora, snapshot).	Bajo	Particionamiento mensual y purga por partición; consultas con pruning.

Riesgo	Impacto	Mitigación
Acoplamiento accidental entre módulos.	Medio	Límites por directorio, contratos por interfaz, revisión de código y análisis estático que prohíbe imports cruzados.
Falla del ERP durante la carga.	Medio	Operación con última carga válida; notificación al administrador; reproceso controlado.
Pérdida de datos por falla de la base de datos.	Alto	Respaldos diarios + WAL; prueba periódica de restauración; replicación si la política lo exige.
Brecha de seguridad por credenciales expuestas.	Alto	Secretos por entorno; conexión_erp solo referencia; logs filtrados; revisión de seguridad en la Etapa 5.
Curva de aprendizaje del equipo con Symfony/Doctrine.	Bajo	Estructura uniforme por módulo; documentación interna; pair programming en la Etapa 0.

Este documento, junto con el SRS y el diseño de base de datos del módulo de Integración, constituye la base técnica del proyecto. Las decisiones pendientes (mecanismo concreto de acceso al ERP, proveedor de CI y de monitoreo de la organización, política de retención de auditoría) se completarán al inicio de la Etapa 0 y se registrarán como ADR en el repositorio.